Find the Promos

Kuku AI Generalist Assignment | Problem 1 | Solution

Pulkit Gulati | Roll No. 23065119 | Civil Engineering, IIT (BHU) Varanasi

Evidence key: [B] Kuku assignment brief; [P] STAGE2_REAL_RUN.md; [R] run_A/eval_report.md, promos.json and run_state.json; [T] run_A/responses/*.success.body; [A] run_A_user_audit.csv; [S] stability_report.md and run_B/run_state.json; [F] WHERE_AI_GOT_IT_WRONG.md, stage3_results(1).json and the source-backed Run A review.

Why this problem and setup

I chose this problem because it addresses Kuku's promo-selection workflow, comes with real material to test, and produces timestamps I can check [B]. I preferred this scope to a legal assistant requiring specialist grounding or an open-ended problem requiring separate validation. I chose local Ollama to run inference on my computer without depending on a paid API service, while retaining control over model configuration. The recorded runs use qwen3:4b-instruct; the model is swappable through configuration [R].

How my tool works

The code parses timed screenplay events and creates a limited menu of candidate time ranges. It checks 20–60 second duration, ordering, cut boundaries, and source-backed opening evidence in the first three seconds [B, P]. Qwen selects and explains promising ranges. The same model judges accepted candidates in a separate pass. The default final score combines 70% model judgments and 30% code-based signals. Model-judged dimensions are hook (0.17), cliffhanger (0.20), standalone clarity (0.15), emotional intensity (0.07), genre fit (0.05), and spoiler avoidance (0.06). Code-based signals are speech within three seconds (0.08), dialogue density (0.06), duration (0.10), and earlier-episode preference (0.06). Judge scores are mapped from 1–10 to 0–1; spoiler risk is reversed for spoiler avoidance. Code removes overlapping cuts and adjusts for diversity before ranking. The workflow also tries prevalidated cross-episode combinations.

I use Streamlit to load input, run the pipeline, inspect ranked results, and record my review. The full-show results cited here are the recorded command-line runs. The tool identifies candidate segments and timestamps; it does not edit, render, or play video. A structural pass checks timing and format; it does not prove a strong hook or cliffhanger.

Working assumptions. The prototype treats screenplay timestamps as corresponding to final-video time, permits candidate windows to cross scene boundaries, and evaluates hooks, cliffhangers, and standalone clarity from screenplay text. These assumptions do not establish that screenplay timing or text perfectly matches the finished video; suggested cuts still require review against the footage.

What the runs produced

From [P], I parsed 50 episodes, 100 scenes, and 3,727 timed events. Run A processed 50/50 episodes: 115 proposals, 100 structurally valid, 85 judged, 15 unjudged, and 81 ranked after overlap filtering [R]. Run B produced 71 ranked and 12 unjudged candidates [S]. Both runs were partial. Structural validity and a ranked score do not establish whether a promo is compelling.

My manual review

I reviewed the top 10 from Run A [A]: Good: ranks 2, 4, 6, 10; Unsure: 3, 7, 8; Bad: 1, 5, 9. The Bad choices ended on a completed line or resolved beat, while ranks 2, 4, and 6 left an outcome, decision, or reveal open. Some Good choices still needed stronger openings or more context. I also found ranks 3 and 9 similar in scene and theme. This is one small, non-random review of screenplay text, not evidence of audience performance.

Where AI and my checks fell short

An early ChatGPT-assisted scan counted 98 scenes because it missed uppercase scene labels in Episode 6; a case-insensitive scan found 100 [P, F]. In an earlier local run, the model judge treated a reply after the cut as if it were inside the clip. I revised the judging input to show only selected source material, but that does not guarantee factual accuracy [F].

My later review found that rank 5's named cliffhanger occurs near the beginning, while the clip ends on a resolved response. For rank 8, the explanation describes an opening strike that ends before the clip begins. Rank 3 ends between timed blocks in a continuing thought. A safe timed boundary can still be a poor editorial cut [A, F].

Run-to-run similarity

The two partial top-10 lists share 3 exact cuts, or 30% of either list. Jaccard similarity is 3 / (10 + 10 - 3) = 3 / 17 = 17.6% [S]. Exact matching does not count similar cuts with different boundaries. Both runs used the same local model at temperature 0.7, but the A/B labels also changed prompts and cache keys; therefore, this comparison does not establish why the results differ. I treat the output as an editor's candidate pool; final selection needs human judgment.

Scale and cost

The table uses Run A's whole-pipeline usage, including cross-episode and judging calls [R, T]. Per-episode figures are Run A totals divided by 50; the 100-episode estimate doubles the totals.

Measure	Run A: 50 episodes	Per episode	Estimate: 100 episodes
Input tokens	534,047	10,681	1,068,094
Output tokens	58,606	1,172	117,212
Model calls	107	2.14	214
Model processing	87.8 min	1.76 min	175.6 min

The processing total is the sum of provider-reported model-call durations, not end-to-end wall time. The estimate assumes similar inputs, hardware, and outcomes; Run A was partial. Local inference has no per-token API charge, but hardware and electricity still cost money.

Proposed, not implemented: queue shows, process episodes across suitable workers, then combine and rank candidates for editorial review. New screenplay formats would need validation.

Limits and AI use

The same model selects and judges, so its scores are not independent validation. The limited candidate search can miss stronger cuts. Overlap removal uses a configured intersection-over-union threshold of 0.6. Greedy re-ranking discourages repeated episodes, five-episode blocks, and literal speaker-label sets; these are heuristics, not verified story arcs, and speaker aliases are not resolved. Script-only analysis cannot assess acting, music, visual impact, or final-video timing, and source checks cannot guarantee that explanations are factually correct, every sentence ends cleanly, or the video matches the screenplay. I would inspect actual footage and test a stronger or independent model with more time; neither is claimed as completed here.

I used ChatGPT for exploration, code, prompts, and debugging. The recorded outputs and limitations are documented in the evidence files listed above.